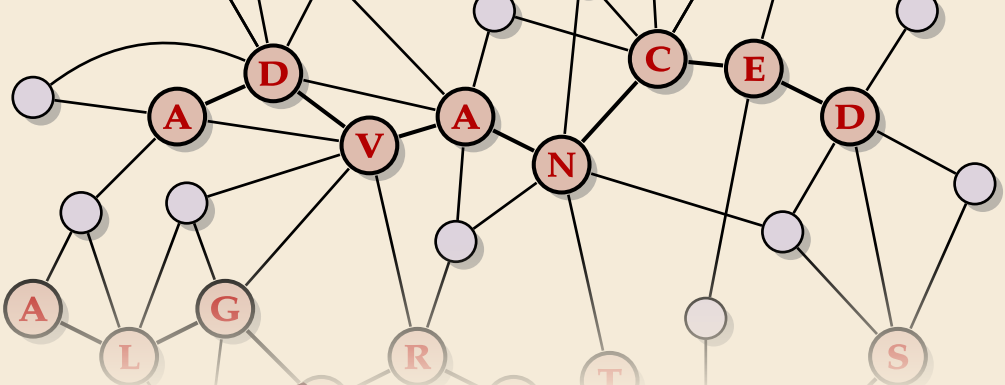



# 4 Fixed-Parameter Algorithms

*Advanced Algorithms · Winter 2026–27*

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
*version* `2026-11-05 10:59 H`

# Outline

# 4 Fixed-Parameter Algorithms


4.1 Fixed-Parameter Tractability

4.2 Depth-Bounded Exhaustive Search I

4.3 Problem Kernels

4.4 Depth-Bounded Search II: Planar Independent Set

4.5 Depth-Bounded Search III: Closest String

4.6 Linear Recurrences & Better Vertex Cover

4.7 Interleaving

# Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* $k$
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of $k$ (and $n$) on running time

  $\rightsquigarrow$ Try to find a parameter $k$ such that
  (1) the problem can be solved efficiently as long as $k$ is small, and
  (2) practical instances have small values of $k$ (even where $n$ gets big).

# Motivation: Satisfiability

**Consider Satisfiability of CNF formula** *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- $k$ = #literals per clause
  - $k \leq 2 \quad \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete
- $k$ = #variables
  - $O(2^k \cdot n)$ time possible (try all assignments)
- $k$ = #clauses?
- $k$ = #literals?
- $k$ = #ones in satisfying assignment
- $k$ = structural property of formula
- for MAX-SAT, $k$ = #optimal clauses to satisfy

# Parameters

## Definition 4.1 (Parameterization)

Let $\Pi = (L, Y, \Sigma)$ be a decision problem. A *parameterization* of $\Pi$ is a polytime computable mapping $\kappa : L \to \mathbb{N}$. ◀

## Definition 4.2 (Parameterized problem)

A *parameterized (decision) problem* is a pair $(\Pi, \kappa)$ of a decision problem $\Pi$ and a parameterization $\kappa$ of $\Pi$. ◀

# Examples

As before: Typically leave encoding implicit.

## Definition 4.3 (p-variables-SAT)

Given: formula boolean $\phi$ (same as before)

Parameter: number of variables

Question: Is there a satisfying assignment $v : [n] \to \{0,1\}$ ? ◀

## Definition 4.4 (p-Clique)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: $k$

Question: $\exists V' \subseteq V \;:\; |V'| \geq k \;\wedge\; \forall u, v \in V' : \{u, v\} \in E$ ? ◀

## Definition 4.5 (p-Vertex-Cover)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: $k$

Question: $\exists S \subseteq V \;:\; |S| \leq k \;\wedge\; \forall e \in E : S \cap e \neq \emptyset$ ? ◀

# Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

**Definition 4.6 (Canonical Parameterization)**

If the instances of a decision problem $\Pi$ are pairs $(x, k)$ with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for $(\Pi, \kappa)$.

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

**Recall:** $\mathit{Threshold}_\Pi$ is the set of pairs $(x, k)$ of all instances $x \in L_I$ that have solutions that are weakly "better" than $k$.

Examples:

- $p$-Clique
- $p$-Vertex-Cover
- $p$-Graph-Coloring
- ...

**Naming convention** for other parameters:
$p$-*clause*-CNF-SAT: CNF-SAT with parameter "number of *clauses*"

## 4.1 Fixed-Parameter Tractability

# Exemplary Running Times of Parameterized Problems

(consider simplest brute-force methods for problems)

- $p$-*variables*-SAT
  - $k$ variables, $n$ length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- $p$-Clique
  - $k$ threshold (clique size); $n$ vertices, $m$ edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$
- $p$-VertexCover
  - $k$ threshold (VC size); $n$ vertices, $m$ edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$
- $p$-GraphColoring
  - $k$ threshold (#colors); $n$ vertices, $m$ edges in graph
  - $\rightsquigarrow$ $k^n$ candidates to check, each takes time $O(m)$
  - $\rightsquigarrow$ Total time $O(k^n \cdot m)$

# FPT Running Time

## Definition 4.7 (fpt-algorithm)

Let $(\Pi, \kappa)$ be a parameterized problem, $\Pi = (L, Y, \Sigma)$.
A (deterministic) algorithm $A$ (with input alphabet $\Sigma$) is a *fixed-parameter tractable algorithm (fpt-algorithm)* w.r.t. $\kappa$ if its running time on $x \in L$ with $\kappa(x) = k$ is at most

$$f(k) \cdot p(|x|) \quad = \quad O\big(f(k) \cdot |x|^c\big)$$

where $p$ is a polynomial of degree $c$ and $f$ is an **arbitrary** computable function. ◀

## Definition 4.8 (FPT)

A parameterized problem $(\Pi, \kappa)$ is *fixed-parameter tractable* if there is an fpt-algorithm w.r.t. $\kappa$ that decides $\Pi$.
The complexity class of all such problems is denoted by FPT. ◀

Intuitively, FPT plays the role of P.

# A First FPT Example

## Theorem 4.9 (p-variables-SAT is FPT)

*p-variables*-SAT $\in$ FPT. ◀

**Proof:**

Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```

Worst case running time: $O(2^k\, n)$ for $n = |\varphi|$.
$2^k$ recursive calls;
base case needs time $O(|\phi|)$ to check whether formula evaluates to *true* ■

. . . but #variables not usually small

# Aren't we all FPT?

**Theorem 4.10 (k never decreases → FPT)**
Let $\Pi = (L, Y, \Sigma)$ be a decidable problem, $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and $\kappa$ a parameterization of $\Pi$ with

$$\forall x \in L : \kappa(x) \geq g(|x|).$$

Then $(\Pi, \kappa) \in \mathsf{FPT}$ (for *any* decidable $\Pi$!). ◀

$g$ weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
$g$ unbounded: $\forall t \; \exists n \; : \; g(n) \geq t$

**Proof:**
$Y$ decidable $\rightsquigarrow$ $\exists$ algorithm $A$ deciding $\Pi$ in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

w.l.o.g. $T$ is weakly increasing and $T(n) \geq n$.

**Idea:** "hide" $T(|x|)$ inside $f(k)$: since $k = \kappa(x) \geq g(|x|)$ and $g$ is unbounded, a large input forces a large parameter.

$\rightsquigarrow$ Define $h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \leq k \,\} \cup \{1\}\big)$.

# Aren't we all FPT? – Proof

**Proof (cont.):**

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \le k \,\} \cup \{1\}\big).$

1. $g$ weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ $h$ well-defined
2. $h$ weakly increasing
3. $g$ computable $\rightsquigarrow$ $h$ computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \ge n$ for all $n$ (as $n$ is in the set for $k = g(n)$)

Now let $x$ be an input, $n = |x|$, $k = \kappa(x) \ge g(n)$. Running $A$ on $x$ takes time

$$T(n) \;\le\; T\big(h(g(n))\big) \;\le\; T\big(h(k)\big) \;=:\; f(k),$$

using (4) and $T$ increasing, then $g(n) \le k$ and $h$, $T$ increasing.

$f$ is computable, so $A$ is an fpt-algorithm for $(\Pi, \kappa)$ (with $p(n) \equiv 1$). ■

# Back to “sensible” parameters

- ⇝ always check if parameter is reasonable (can be expected to be small)
  - if not, FPT might not even mean in NP!
- but now, for some positive examples!

## 4.2 Depth-Bounded Exhaustive Search I

# FPT Design Pattern

- The simplest FPT algorithms use exhaustive search
- but with a search tree bounded by $f(k)$

- bruteforceSat was a typical example!
- does this work on other problems?

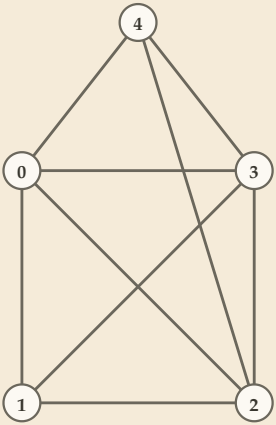
# Depth-Bounded Search for Vertex Cover

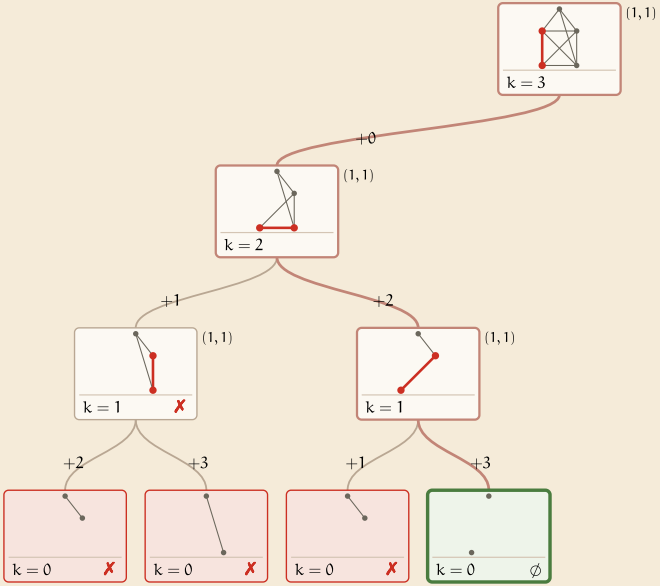
Let's try $p$-VertexCover.
Key insight: for every edge $\{v, w\}$, any vertex cover must contain $v$ or $w$

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      if E == ∅ then return ∅
3      if k == 0 then return NOT_POSSIBLE  // truncate search
4      Choose {v, w} ∈ E (arbitrarily)
5      for u in {v, w} do:
6          G_u := (V \ {u}, E \ {{u, x} ∈ E})  // Remove u from G
7          C_u := simpleFptVertexCover(G_u, k − 1)
8      if C_v == NOT_POSSIBLE then return C_w ∪ {w}
9      if C_w == NOT_POSSIBLE then return C_v ∪ {v}
10     if |C_v| ≤ |C_w| then return C_v ∪ {v} else return C_w ∪ {w}
```

- Does not need explicit checks of solution candidates!
- runs in time $O\left(2^k(n+m)\right)$ $\rightsquigarrow$ fpt-algorithm for $p$-Vertex-Cover

# Depth-Bounded Search for Vertex Cover – Example



# Guessing the parameter

- Note: Previous algorithm only uses $k$ to *truncate* branches.

$\rightsquigarrow$ We can *guess* a $k$ and it still works

$\rightsquigarrow$ Try all $k$!

```
1 procedure vertexCoverBfs(G = (V, E))
2     for k := 0, 1, ..., |V| do
3         C := simpleFptVertexCover(G, k)
4         if C ≠ NOT_POSSIBLE return C
```

- Running time: $\sum_{k'=0}^{k} O(2^{k'}(n+m)) \;=\; O(2^k(n+m))$

$\rightsquigarrow$ For exponentially growing cost, trying all values up to $k$ costs only constant factor more

## 4.3 Problem Kernels

# Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set $x$ to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case . . .
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers
- Trivial example for VertexCover

  include other endpoint in solution

  Remove vertices of degree 0 or 1. (never needed as part of optimal VC)
- Here: reduction rules that provably shrink an instance to size $g(k)$

# Buss's Reduction Rule for VC

- Given a $p$-VertexCover instance $(G, k)$

  **"deg > $k$" Rule:** If $G$ contains vertex $v$ of degree $\deg(v) > k$, include $v$ in potential solution and remove it from the graph.

- Can apply this simultaneously to degree $> k$ vertices.
- Either rule applies, or all vertices bounded degree(!)

# Kernels

## Definition 4.11 (Kernelization)

Let $(\Pi, \kappa)$ be a parameterized problem, $\Pi = (L, Y, \Sigma)$.
A function $K : L \to L$ is a *kernelization* of $(\Pi, \kappa)$ if it maps every instance $x \in L$ with $k = \kappa(x)$ to an instance $x' = K(x)$ so that

1. (self-reduction) $x \in Y \iff x' \in Y$
2. (polytime) $K$ is computable in polytime.
3. (kernel-size) $|x'| \leq g(k)$ and $\kappa(x') \leq g(k)$ for some computable function $g$

We call $x'$ the *(problem) kernel* of $x$ and $g$ the *size of the problem kernel*. ◀

# Buss's Kernel

**Buss's Reduction for Vertex Cover:** (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree $0$ and $1$ vertices

## Theorem 4.12 (Buss's Reduction is Kernelization)

Buss' reduction yields a kernelization for $p$-Vertex-Cover with kernel size $O(k^2)$. ◀

**Proof:**
After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.
(Note that the rule might reduce the parameter $k$).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.
If $m > k^2$, we output a trivial No-instance (e. g., a $K_{k+1}$ a complete graph on $k+1$ vertices).
If $m \leq k^2$, then the input size is now bounded by $g(k) = 2k^2$. ■

# FPT iff Kernelization

**Theorem 4.13 (FPT ↔ kernel)**

Let $(\Pi, \kappa)$ be a parameterized problem with $\Pi$ decidable.
$(\Pi, \kappa)$ is fixed-parameter tractable if and only if there is a kernelization of $(\Pi, \kappa)$.

**Proof:**

# FPT iff Kernelization [2]

**Proof (cont.):**

# Max-SAT Kernel

**Theorem 4.14 (Kernel for Max-SAT)**

$p$-Max-SAT has a problem kernel of size $O(k^2)$ which can be constructed in linear time. ◀

**Proof:**

# Max-SAT Kernel [2]

**Proof (cont.):**

# Max-SAT Kernel [3]

**Proof (cont.):**

**Corollary 4.15**

$p$-Max-SAT $\in$ FPT

# 4.4 Depth-Bounded Search II: Planar Indep. Set

# Deeper results (towards more shallow trees)

- Our previous examples of depth-bounded search were basically brute force
- Here we will see two more examples that exploit the problem structure in more interesting ways

- Recall: IndependentSet
- IndependentSet is NP-hard on planar graphs even with vertex degrees at most 3
- We will see: general problem $p$-Independent-Set likely not in FPT.

# Independent Set on Planar Graphs

### Definition 4.16 ($p$-Planar-Independent-Set)

Given: a *planar* graph $G = (V, E)$ and $k \in \mathbb{N}$
Parameter: $k$
Question: $\exists V' \subset V \quad : \quad |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$ ? ◀

### Theorem 4.17 (Depth-Bounded Search for Planar Independent Set)

$p$-Planar-Independent-Set is in FPT and can be solved in time $O(6^k n)$. ◀

# Elementary Knowledge on Planar Graphs

### Theorem 4.18 (Euler's formula)

In any finite, *connected* (embedded) planar graph $G$ with $n$ vertices, $m$ edges, and $f$ faces holds $n - m + f = 2$. ◀

### Corollary 4.19

A *simple* planar graph $G$ on $n \geq 3$ nodes has $m \leq 3n - 6$ edges.
The average degree in $G$ is $< 6$. ◀

# Depth-Bounded Search for Planar Independent Set

```
1  procedure planarIndependentSet(G = (V, E), k):
2      if k == 0 then return ∅
3      if k > |V| then return NOT_POSSIBLE // truncate search
4      Choose v ∈ V with minimal degree; let w_1, ..., w_d be v's neighbors
5      // By planarity, we know d ≤ 5.
6      for u in {v, w_1, ..., w_d} do
7          D := {u} ∪ N(u)
8          G_u := (V \ D, E \ {{x, y} ∈ E : x ∈ D}) // Delete u and its neighbors
9          I'_u := planarIndependentSet(G_u, k − 1)
10         if I'_u ≠ NOT_POSSIBLE then return {u} ∪ I'_u
11     return NOT_POSSIBLE
```

# Summary Planar Independent Set

- planarIndependentSet will often be faster than $O(6^k n)$
- works unchanged in $O((d+1)^k n)$ time for any degeneracy-$d$ graph

every (induced) subgraph has vertex of degree at most $d$

# 4.5 Depth-Bounded Search III: Closest String

# Closest String

### Definition 4.20 ($p$-Closest-String)

Given: S set of $m$ strings $s_1, s_2, \ldots, s_m$ of length $L$ over alphabet $\Sigma$ and a $k \in \mathbb{N}$.
Parameter: $k$
Question: Is there a string $s$ for which $d_H(s, s_i) \leq k$ holds for all $i = 1, \ldots, m$? ◀

# Dirty Columns

### Definition 4.21 (Dirty Column)

A column of the $m \times L$ matrix corresponding to $m$ strings of length $L$ is called *dirty* if it contains at least 2 different symbols. ◀

### Lemma 4.22 (Many Dirty Columns → No)

Let an instance to Closest-String with $m$ strings of length $L$ and parameter $k$ be given.
If the corresponding $m \times L$ matrix contains more than $m \cdot k$ dirty columns, then no solution for the given instance exists. ◀

# Depth-Bounded Search for Closest String

```
1  procedure closestStringFpt(s, d):
2      if d < 0 then return NOT_POSSIBLE
3      if d_H(s, s_i) > k + d for an i ∈ {1, ..., m} then
4          return NOT_POSSIBLE
5      if d_H(s, s_i) ≤ k for all i = 1, ..., m then return s
6      Choose i ∈ {1, ..., m} arbitrarily with d_H(s, s_i) > k
7          P := {p : s[p] ≠ s_i[p]}
8          Choose arbitrary P' ⊆ P with |P'| = k + 1
9          for p in P' do
10             s' := s
11             s'[p] := s_i[p]
12             s_ret := closestStringFpt(s', d − 1)
13             if s_ret ≠ NOT_POSSIBLE then return s_ret
14     return NOT_POSSIBLE
```

- initial call closestStringFpt($s_1, k$)

# Too Much Dirt

**Lemma 4.23 (Pair Too Different → No)**
Let $S = \{s_1, s_2, \ldots, s_m\}$ a set of strings and $k \in \mathbb{N}$. If there are $i, j \in \{1, \ldots, m\}$ with $d_H(s_i, s_j) > 2k$, then there is no string $s$ with $\max_{1 \leq i \leq m} d_H(s, s_i) \leq k$. ◀

# Depth-Bounded Search for Closest String

## Theorem 4.24 (Search Tree for Closest String)

There is a search tree of size $O(k^k)$ for problem $p$-Closest-String. ◀

## Corollary 4.25 (Closest String is FPT)

$p$-Closest-String can be solved in time $O(mL + mk \cdot k^k)$. ◀

- preprocessing ($O(mL)$ time)
  - ignore any clean columns
  - reject if more than $mk$ dirty columns

⇝ effective string length after preprocessing is $L' \leq mk$

- call closestStringFpt($s_1, k$)
  - maintain $d_H(s, s_i)$ in an array
    - ⇝ checking any distance $d_H(s, s_i)$ takes $O(1)$ time
    - before and after recursive call, update array to reflect $d_H(s', s_i)$
      Single character changed, so update only needs to check single position
    - ⇝ Can maintain distances in $O(m)$ time per recursive call
  - $P'$ can be computed in $O(mk)$ time

## 4.6 Linear Recurrences & Better Vertex Cover

# A Better Algorithm for Vertex Cover

Recall: Branching on endpoints of $k$ edges gives search space of size $2^k$ for Vertex-Cover.
Can we do better?

**Idea:** Enlarge base case with "easy inputs"

Here: Consider graphs $G$ with $\deg(v) \leq 2$ for all $v \in V(G)$.

# Depth-Bounded Search for Vertex Cover

```
1  procedure betterFptVertexCover(G = (V, E), k):
2      if E = ∅ then return ∅
3      if k ≤ 0 then return NOT_POSSIBLE  // truncate search
4      if all nodes have degree ≤ 2 then
5          Find connected components of G
6          for each component G_i do
7              Fill C_i by picking every other node,
8              starting with the neighbor of a degree-one node if one exists
9          C := ⋃ C_i
10         if |C| ≤ k then return C else return NOT_POSSIBLE
11     Choose v with maximal degree, let w_1, ..., w_d be its neighbors // d ≥ 3
12     for D in {{v}, {w_1, ..., w_d}} with |D| ≤ k do:
13         G_D := G[V \ D] // Remove D from G
14         C'_D := betterFptVertexCover(G_D, k − |D|)
15         if C'_D ≠ NOT_POSSIBLE then C_D := D ∪ C'_D
16     return smallest C_D found, or NOT_POSSIBLE if none
```

*How to analyze running time of betterFptVertexCover?*

# Analysis of betterFptVertexCover

worst case running time

- never have all degrees $\leq 2$
- always need both recursive calls (until base case)
- ignore that graph gets smaller

$T_0 = \Theta(1)$
$T_k = \Theta(|V| + |E|) + T_{k-3} + T_{k-1}$

If we only number of base cases $B_n$, we obtain $T_n = O(B_n n^2)$

$B_0 = 1,\ B_1 = 1,\ B_2 = 1$
$B_k = B_{k-3} + B_{k-1} \qquad (k \geq 3)$

# Solving Linear Recurrences

**Example:** $B_0 = B_1 = B_2 = 1, \qquad B_k = B_{k-1} + B_{k-3} \quad (k \geq 3)$

**Tool:** the *ordinary generating function* $B(z) := \sum_{k \geq 0} B_k z^k$

(a formal power series; $[z^k]\, B(z) = B_k$ extracts the coefficient)

Translate the recurrence into an equation for $B(z)$:

$$\begin{aligned} B(z) &= 1 + z + z^2 + \sum_{k \geq 3} \big(B_{k-1} + B_{k-3}\big) z^k \\ &= 1 + z + z^2 + z \sum_{k \geq 3} B_{k-1} z^{k-1} + z^3 \sum_{k \geq 3} B_{k-3} z^{k-3} \\ &= 1 + z + z^2 + z\big(B(z) - B_0 - B_1 z\big) + z^3 B(z) \\ &= 1 + z\, B(z) + z^3 B(z). \end{aligned}$$

$$B(z)\big(1 - z - z^3\big) = 1 \qquad \rightsquigarrow \qquad \boxed{B(z) = \frac{1}{1 - z - z^3}}$$

# Solving Linear Recurrences [2]

Let $z_0, z_1, z_2$ be the roots of $1 - z - z^3$ (all distinct here): $z_0 \approx 0.68233$, $z_{1,2} \approx -0.34116 \pm 1.16154\,i$.

**Partial fractions:** $B(z) = \dfrac{1}{1-z-z^3} = \dfrac{A_0}{z-z_0} + \dfrac{A_1}{z-z_1} + \dfrac{A_2}{z-z_2}$ for constants $A_0, A_1, A_2$.

Each term is a geometric series:

$$\frac{A_j}{z-z_j} = \frac{-A_j/z_j}{1-z/z_j} = -\frac{A_j}{z_j}\sum_{k\ge 0}\left(\frac{1}{z_j}\right)^k z^k \qquad \rightsquigarrow \qquad [z^k]\,\frac{A_j}{z-z_j} = -\frac{A_j}{z_j}\left(\frac{1}{z_j}\right)^k.$$

$$B_k = [z^k]\,B(z) = -\frac{A_0}{z_0}\left(\frac{1}{z_0}\right)^k - \frac{A_1}{z_1}\left(\frac{1}{z_1}\right)^k - \frac{A_2}{z_2}\left(\frac{1}{z_2}\right)^k = \Theta\Big(\big(\tfrac{1}{z_0}\big)^k\Big),$$

since $|z_0| < |z_1| = |z_2|$ (the root closest to $0$ dominates).

$\rightsquigarrow$ $B_k = \Theta\big(1.4656^k\big)$ with $1/z_0 \approx 1.4656$

Note: $1/z_0$ is the *largest* root of the *characteristic polynomial* $z^3 - z^2 - 1$ (substitute $z \mapsto 1/z$).

# Solving Linear Recurrences – Result

## Theorem 4.26 (Linear Recurrences)

Let $d_1, \ldots, d_i \in \mathbb{N}$ and $d = \max d_j$.
The solution to the *homogeneous linear recurrence equation*

$$T_n \quad = \quad T_{n-d_1} + T_{n-d_2} + \cdots + T_{n-d_i}, \qquad (n \geq d)$$

is always given by

$$T_n \quad = \quad \sum_{\ell} \sum_{j=0}^{\mu_\ell - 1} c_{\ell,j}\, z_\ell^n\, n^j$$

where we sum over all roots $z_\ell$ of multiplicity $\mu_\ell$ of the so-called *characteristic polynomial* $z^d - z^{d-d_1} - z^{d-d_2} \cdots - z^{d-d_i}$.
The $d$ coefficients $c_{\ell,j}$ are determined by the $d$ initial values $T_0, T_1, \ldots, T_{d-1}$. ◀

## Corollary 4.27

$T_n = O(z_0^n n^d)$ for $z_0$ the root of the characteristic polynomial with *largest absolute value*. ◀

# Analysis of betterFptVertexCover [2]

$T_0 = \Theta(1)$,
$T_k = \Theta(n+m) + T_{k-3} + T_{k-1}$ $(n = |V|, m = |E|)$

Counting only leaves $B_k$ of the search tree (internal nodes $\leq$ leaves): $T_k = O\big(B_k \cdot (n+m)\big)$

$B_0 = 1,\ B_1 = 1,\ B_2 = 1$
$B_k = B_{k-3} + B_{k-1}$ $(k \geq 3)$

$\rightsquigarrow$ $\vec{d} = (1, 3)$; characteristic polynomial $z^3 - z^2 - 1$
roots at $z_0 \approx 1.4656$ and $z_{1,2} \approx -0.2328 \pm 0.7926i$

**Theorem 4.28 (Depth-Bounded Search for Vertex Cover)**
$p$-Vertex-Cover can be solved in time $O(1.4656^k n^2)$. ◀

# 4.7 Interleaving

# Motivation

Up to now, considered two-phase algorithms

1. Reduction to problem kernel
2. Solve kernel by depth-bounded exhaustive search

Idea: Apply kernelization *in each recursive step*.

# (Extreme) Example: Vertex Cover with large-degree rule

- As a (slightly artificial) example, consider only using the simple reduction rule

  **"deg > $k$" Rule:** If $G$ contains vertex $v$ of degree $\deg(v) > k$, include $v$ in potential solution and remove it from the graph.

- **Algorithm A:**
  1. Apply $\deg > k$ rule until saturation
  2. Call simpleFptVertexCover (recursively branch over arbitrary edge)
- **Algorithm B:** Same, interleaved:
  - Modified simpleFptVertexCover
  - Before choosing each new edge to branch on, apply $\deg > k$ rule.

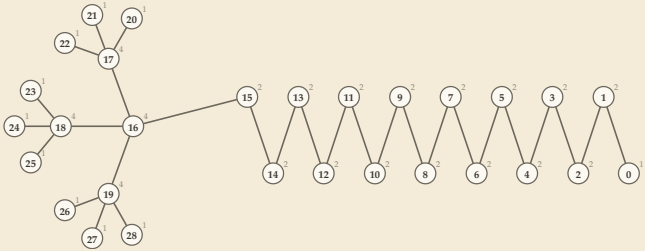
# SimpleFptVertexCover Interleaved

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      // nothing
3      // new
4      // on
5      // this
6      // side
7      if E == ∅ then return ∅
8      if k == 0 then return NOT_POSSIBLE
9      Choose {v, w} ∈ E (arbitrarily)
10     for u in {v, w} do:
11         G_u := G[V \ {u}]
12         C_u := simpleFptVertexCover(G_u, k − 1)
13     if C_v == NOT_POSSIBLE then return C_w ∪ {w}
14     if C_w == NOT_POSSIBLE then return C_v ∪ {v}
15     if |C_v| ≤ |C_w| then
16         return C_v ∪ {v}
17     else
18         return C_w ∪ {w}
```

```
1  procedure simpleInterleavedVC(G = (V, E), k):
2      C := ∅
3      while k > 0 ∧ ∃v ∈ V : deg(v) > k
4          G := G[V \ {v}] // Remove v
5          C := C ∪ {v}
6          k := k − 1
7      if E == ∅ return C
8      if k == 0 return NOT_POSSIBLE
9      Choose {v, w} ∈ E (arbitrarily)
10     for u in {v, w} do:
11         G_u := G[V \ {u}]
12         C_u := C ∪ simpleInterleavedVC(G_u, k − 1)
13     if C_v == NOT_POSSIBLE then return C_w ∪ {w}
14     if C_w == NOT_POSSIBLE then return C_v ∪ {v}
15     if |C_v| ≤ |C_w| then
16         return C_v ∪ {v}
17     else
18         return C_w ∪ {w}
```

# Comparison on Lollipop Flowers

Consider family of graphs $G_k$ "Lollipop Flowers":
"head" vertex with $k-2$ stars of $k-2$ leaves each attached + "tail" of $3k+1$ vertex path




$n = |V(G_k)| = (k-2)(k-1)+1 + 3k+1 = k^2+4$

**Algorithm A**

$\deg > k$ rule does nothing
search space remains $2^k$
Answer No after exploring all branches

$\rightsquigarrow$ time $\Theta(2^k k^2)$

**Algorithm B**

initially same (no reduction)
after $2$ edges removed from tail, parameter $k-2$
vertices in head have degree $k-1$
Output No (parameter $0$, but tail edges left)

$\rightsquigarrow$ time $\Theta(k^2)$

# Setting for Interleaving

*Can we prove a general speedup?*

**Assumptions:** (more restrictive than general kernelization!)

- $K$ kernelization that
  - produces *kernel of size* $\leq q(k)$ for $q$ a *polynomial*
  - in time $\leq p(n)$ for $p$ a polynomial
- Branch in depth-bounded search tree
  - into $i$ subproblems with branching vector $\vec{d} = (d_1, \ldots, d_i)$ (i. e., parameter in subproblems $k - d_1, \ldots, k - d_i$)
  - Branching is computed in time $\leq r(n)$ for $r$ a polynomial

$\rightsquigarrow$ search space has size $O(\alpha^k)$.

$\rightsquigarrow$ Running time of two-phase approach on input $x$ with $n = |x|$ and $k = \kappa(x)$:

$$O\Big(p(n) \;+\; r\big(q(k)\big) \cdot \alpha^k\Big)$$

# With Interleaving

Generic interleaving:

1 **if** $|I| > c \cdot q(k)$ **then**
2 $\quad (I, k) := (I', k')$ where $(I', k')$ forms a problem kernel *// Conditional Reduction*
3 **end**
4 replace $(I, k)$ with $(I_1, k - d_1), (I_2, k - d_2), \ldots, (I_i, k - d_i)$ *// Branching*

$\rightsquigarrow$ Running time of interleaved approach on input $x$ with $n = |x|$ and $k = \kappa(x)$ is at most $T_k$:

$$T_\ell \quad = \quad T_{\ell - d_1} + \cdots + T_{\ell - d_i} \; + \; p\big(q(\ell)\big) + r\big(q(\ell)\big)$$

Compare to non-interleaved version:

$$T_\ell \quad = \quad T_{\ell - d_1} + \cdots + T_{\ell - d_i} \; + \; r\big(q(k)\big)$$

Here the inhomogeneous term is constant w.r.t. $\ell$, but depends on $k$
$\rightsquigarrow$ cannot ignore constant factors

# Analysis of interleaved betterFptVertexCover [1]

Consider betterFptVertexCover from before, but with $\deg > k$ rule added.

- Initial call has unbounded $n$ and $m$; after applying degree $0, 1, > k$ rules (in $O(n + m)$ time) size of graph $n + m = O(k^2)$
- interleaving $\rightsquigarrow$ graph also bounded recursively (in terms of new $k$)
- Recursive worst-case time after first reduction:
  $T_0 = \Theta(1)$
  $T_k = O(k^2) + T_{k-3} + T_{k-1}$

# Inhomogenous Linear Recurrences

**Example:** $T_0 = T_1 = T_2 = 1, \qquad T_k = k^2 + T_{k-1} + T_{k-3} \quad (k \geq 3)$

Same translation as before, $T(z) \coloneqq \sum_{k\geq 0} T_k z^k$:

$$\begin{aligned} T(z) &= 1 + z + z^2 + \sum_{k\geq 3} \big(k^2 + T_{k-1} + T_{k-3}\big) z^k \\ &= 1 + z\big(T(z) - 1 - z\big) + z^3 T(z) + \sum_{k\geq 3} k^2 z^k. \end{aligned}$$

The inhomogeneous part is a *rational function with poles only at* $z = 1$:

$$\sum_{k\geq 0} z^k = \frac{1}{1-z}, \qquad \sum_{k\geq 1} k z^{k-1} = \frac{d}{dz}\frac{1}{1-z} = \frac{1}{(1-z)^2}, \qquad \sum_{k\geq 2} k(k-1) z^{k-2} = \frac{2}{(1-z)^3},$$

so $\sum_{k\geq 3} k^2 z^k = \dfrac{P(z)}{(1-z)^3}$ for a polynomial $P$.

# Inhomogenous Linear Recurrences [2]

Solving for $T(z)$:

$$T(z) \;=\; \frac{Q(z)}{(1-z-z^3)\,(1-z)^3} \qquad \text{for a polynomial } Q.$$

$\rightsquigarrow$ Poles of $T(z)$: the roots $z_0, z_1, z_2$ of $1-z-z^3$ **and** $z=1$ (triple).

Dominant singularity is still $z_0 \approx 0.68 < 1$ $\rightsquigarrow$ $T_k = \Theta\big((1/z_0)^k\big) = \Theta(B_k)$.

General principle: an inhomogeneity $f_k$ whose GF has no singularity inside $|z| \le |z_0|$ does not change the growth rate.

# Inhomogenous Linear Recurrences Summary

## Theorem 4.29 (Linear Recurrences II)

Let $d_1, \ldots, d_i \in \mathbb{N}$, $d = \max d_j$, and let $z_0 > 1$ be the real root of largest absolute value of $z^d - \sum_{j=1}^{i} z^{d-d_j}$. Consider the ***in****homogeneous linear recurrence*

$$T_n \quad = \quad T_{n-d_1} + T_{n-d_2} + \cdots + T_{n-d_i} + \boldsymbol{f_n}, \qquad (n \geq d)$$

with initial values $T_0, \ldots, T_{d-1} \in \mathbb{R}_{>0}$ and $(f_n)_{n\in\mathbb{N}}$ a sequence of positive numbers with $f_n = O\big((z_0 - \varepsilon)^n\big)$ for some fixed $\varepsilon > 0$ (e. g., any polynomial).
Then $T_n = O(T_n^0)$ where $T_n^0$ is defined as $T_n$ with $f_n \equiv 0$. ◀

# A Little Excursion: Singularity Analysis

**General strategy:** use generating functions for asymptotic approximations

| **Sequence Land** | | **Generating Function Land** |
|---|---|---|
| ▶ number sequence $(a_n)_{n\geq 0}$ | $\longrightarrow$ | ▶ (ordinary) generating function $A(z) = \sum_{n\geq 0} a_n z^n$ |
| ▶ recurrence equation | $\longrightarrow$ | ▶ (functional) equation for $A(z)$ |
| | | $\downarrow$ solve, simplify (e. g., partial fractions) |
| | | $\rightsquigarrow$ closed form for $A(z)$ |
| ▶ closed form for $a_n$ | $\longleftarrow$ | ▶ exact coefficients $[z^n]A(z)$ |
| | | OR approximate $A(z)$ near its *dominant singularity* |
| ▶ *asymptotic* approximation $a_n = z_0^{-n} n^{\alpha-1}(1 \pm O(n^{-1}))$ | $\longleftarrow$ transfer thms | $\rightsquigarrow$ *singular expansion* at $z = z_0$ $A(z) = f(z) \pm O((1 - z/z_0)^{-\alpha})$ |

# O-Transfer

**Theorem 4.30 (Transfer-Theorem of Singularity Analysis)**

Assume $f(z)$ is $\Delta$-analytic and admits the singular expansion

$$f(z) \;=\; g(z) \;\pm\; O\big((1-z)^{-\alpha}\big) \qquad (z \to 1)$$

with $\alpha \in \mathbb{R}$. Then

$$[z^n]\, f(z) \;=\; [z^n] g(z) \;\pm\; O\big(n^{\alpha-1}\big) \qquad (n \to \infty).$$

# Possible Extensions

- (constant) coefficients $c_j \cdot T_{n-d_j}$ in recurrence
  $\rightsquigarrow$ different characteristic polynomial, same ideas
- *any* recurrence that leads to a representation of the generating function as a *singular expansion* around the dominant singularity.

$$\begin{aligned} f(z) &= c(1-z/z_0)^{-m} \pm O((1-z/z_0)^{-m+1}) \qquad (z \to z_0) \\ \rightsquigarrow [z^n]\, f(z) &= \frac{c}{(m-1)!} z_0^{-n} n^{m-1} \cdot \left(1 \pm O(n^{-1})\right) \qquad (n \to \infty) \end{aligned}$$

- other powers $\alpha$ in $1/(1-z)^\alpha$:

$$[z^n] \frac{1}{(1-\frac{z}{z_0})^\alpha} = \frac{z_0^{-n} n^{\alpha-1}}{\Gamma(\alpha)} \left(1 \pm O(n^{-1})\right) \qquad (n \to \infty) \qquad \begin{matrix} -\alpha \notin \mathbb{N}_0 \\ z_0 > 0 \end{matrix}$$

- much more! $\rightsquigarrow$ *analytic combinatorics*

# Analysis of interleaved betterFptVertexCover [2]

- $T_0 = \Theta(1)$
  $T_k = O(k^2) + T_{k-3} + T_{k-1}$

$\rightsquigarrow$ $T_k = O(1.4656^k)$ (same characteristic polynomial)

- Total time: $O(1.4656^k + n + m)$

- The current record is $O^*(1.25284^k)$ time:

  **D. G. Harris, N. S. Narayanaswamy**: *A faster algorithm for Vertex Cover parameterized by solution size,* STACS **2024**

  before that, for more than a decade: $O(1.2738^k + kn)$

  **J. Chen, I. A. Kanj, G. Xia**: *Improved upper bounds for vertex cover,* Theoretical Computer Science 411(40–42), 3736–3756 **2010**

# Summary

- Strategies for fpt algorithms
  - Use parameter to bound depth of exhaustive search
  - Use problem specific reduction rules to shrink input $\rightsquigarrow$ kernel(ization)s
- analysis of exact exponential searches often reduces to linear recurrences
  - generating functions!
- more clever branching reduces exponent of search space
- interleaving kernelization and exhaustive search improves polynomial parts